

Elective Surgery Scheduling

Interval-based CP-SAT model for weekly OR planning

Hector Bonilla

Agenda

1 Problem & Case Study

2 CP-SAT Model

3 Data & Results

4 Conclusions

20-minute flow: problem context and Portuguese case study → CP-SAT formulation → experimental results → conclusions. The MIP comparison model, full variable reference, pipeline diagram, and deployment notes are in the appendix.

The scheduling problem

One hospital, one week. An elective waiting list always bigger than the week's OR capacity.

- **Decision per case:** which room, which day, what start time, which surgeon – or defer to next week.
- **Feasibility:** rooms, surgeons, shared equipment, and downstream recovery beds are all finite and shared.
- **Fairness:** clinical urgency, not FIFO order, should determine priority – an overdue patient must not wait behind a routine one indefinitely.
- **Cost:** OR time runs $\sim$ \$30–40/min once staffing and overhead are amortised (Macario, 2010). An idle minute today cannot be banked for tomorrow.
- **Scope:** *advance scheduling* – assigning cases to days and rooms before the week starts (Cardoen et al., 2010) – distinct from same-day sequencing or emergency disruption handling.

Case study: Portugal's SIGIC system

SIGIC (*Sistema Integrado de Gestão de Inscritos para Cirurgia*) – Portugal's national elective surgical waiting-list management system, established by **Portaria n.º 45/2008** (Diário da República).

Every elective case is assigned one of **four clinical priority tiers**, each with a legislated **maximum acceptable wait**:

Priority	Clinical meaning	Max wait	Model parameter
1	Routine elective	270 days	$wl_1^{max} = 270$
2	Elevated urgency	60 days	$wl_2^{max} = 60$
3	Urgent	15 days	$wl_3^{max} = 15$
4	Emergent add-on (run this week)	3 days	$wl_4^{max} = 3$

These four numbers are the model's wl_p^{max} parameters – taken directly from a real, legislated, annually audited policy. The same tier structure appears in the UK NHS RTT targets and Canadian provincial wait-time benchmarks.

Case study: how SIGIC enforces its own deadlines

SIGIC is not just a published target – it is run by **ACSS** (*Administração Central do Sistema de Saúde*), the central authority that all NHS-affiliated hospitals report into through a shared national registry, so every patient's wait clock is tracked centrally, not hospital-by-hospital.

The mechanism that gives the deadline teeth: the surgical voucher (*Cheque Cirúrgico*). When a hospital cannot operate within a case's tier deadline, the patient gains the right to be referred to an *alternative* provider – public, private, or social-sector – inside the SIGIC network, paid for by the NHS rather than the originating hospital.

Case study: how SIGIC enforces its own deadlines

SIGIC is not just a published target – it is run by **ACSS** (*Administração Central do Sistema de Saúde*), the central authority that all NHS-affiliated hospitals report into through a shared national registry, so every patient's wait clock is tracked centrally, not hospital-by-hospital.

The mechanism that gives the deadline teeth: the surgical voucher (*Cheque Cirúrgico*). When a hospital cannot operate within a case's tier deadline, the patient gains the right to be referred to an *alternative* provider – public, private, or social-sector – inside the SIGIC network, paid for by the NHS rather than the originating hospital.

Why this is the right real-world analog for w_c

A breach under SIGIC is not just a worse statistic – it becomes a *real, billable cost* to the institution that missed the deadline. That is exactly the role the non-scheduling penalty w_c plays in this model's objective (Term 3): it is sized to be large enough that the optimiser treats a breach as something to actively avoid, not a number to merely report after the fact.

Case study: the 2016 CHLN audit

Marques & Captivo (2015) audited the **Centro Hospitalar Lisboa Norte (CHLN)** waiting list against SIGIC targets:

- **~7,400** patients on the waiting list at audit time.
- **16%** had *already* breached their tier's maximum-wait deadline.
- Average overrun among breached cases: **147 days** hospital-wide.
- Worst service – Neurosurgery: **~261 days** average overrun among its breached patients.

Case study: the 2016 CHLN audit

Marques & Captivo (2015) audited the **Centro Hospitalar Lisboa Norte (CHLN)** waiting list against SIGIC targets:

- ~**7,400** patients on the waiting list at audit time.
- **16%** had *already* breached their tier's maximum-wait deadline.
- Average overrun among breached cases: **147 days** hospital-wide.
- Worst service – Neurosurgery: ~**261 days** average overrun among its breached patients.

Why these numbers shape this model's objective

A planner maximising throughput alone can report excellent utilisation while quietly reproducing this failure: scheduling routine cases, deferring overdue ones indefinitely, because a FIFO queue or an unweighted objective cannot distinguish them. The **non-scheduling penalty** w_c exists specifically to make breach costs *explicit, priority-weighted, and dominant* in the objective – so the optimiser cannot accidentally reproduce the audit finding.

What the model covers – and what it doesn't

Modelled:

- Case → room → time → surgeon assignment
- Room / service rosters (block schedule)
- Surgeon availability, daily and weekly limits
- One shared imaging unit (C-arm) with capacity
- Recovery / ICU bed pool (multi-day stay)
- Priority-4 emergency hard lock to day 1
- Paediatric-block carve-out rule

Deliberately excluded:

- Stochastic durations
- Same-day disruptions / emergency adds
- Nurse / anaesthetist rostering
- Sequence-dependent OR cleaning
- Multi-week rolling horizon
- Emergency capacity reserve sizing

Key assumptions (full justification: FORMULATION.md §2):

- Deterministic durations (historical median per procedure).
- Turnover bucketed by case duration: 15 / 25 / 40 min.
- Surgeons as the binding staffing resource (nurses assumed bundled with the rostered room).
- Constant bed capacity across the week; explicit overflow charge for stays crossing the horizon.

Why CP-SAT and not a MIP?

Core argument – counting $\neq$ non-overlap for shared resources

“Total equipment-use minutes today $\leq$ capacity” certifies that durations *fit*. It does *not* certify they don’t *collide in time*. For a single room these coincide; for a resource shared across multiple rooms they stop coinciding.

MIP (day-bucket): count C-arm uses per day ≤ 1 .
Two non-overlapping C-arm cases on the same day
fail this check even if they never run concurrently $\Rightarrow$
feasible schedules are forbidden.

CP-SAT: AddCumulative checks literal time overlap.
Two sequential C-arm cases pass if they never run
concurrently $\Rightarrow$ larger, correct feasible region $\Rightarrow$
strictly better schedules.

Structural reason – C11 (recovery/ICU beds): the constraint “bed_c starts on the day c is operated” requires a variable holding “day of surgery” as a concrete *value*. A day-bucket MIP has no such variable – C11 cannot be expressed in that formulation at all, regardless of solver performance. [▶ Full MIP \(appendix\)](#)

Sets and parameters

Key parameters

Sets and indices

Symbol	Meaning
$c \in C$	Surgical cases
$d \in D$	Days $\{1, \dots, 5\}$
$r \in R$	Operating rooms
$h \in H$	Surgeons
$e \in E$	Shared equipment types
$D_c \subseteq D$	Eligible days for c

Symbol	Meaning
$t_c^{op}, t_c^{clean}, t_c^{tot}$	Duration, turnover, total
k_{dr}, k_{hd}, k_h	Room / surgeon daily / weekly cap
$p_c \in \{1..4\}$	Priority tier (SIGIC)
wl_c	Days waited at planning date
wl_p^{max}	Max wait for tier p : 270/60/15/3
$dd_c = wl_p^{max} - wl_c$	Slack (< 0 = overdue)
$\mu_p = wl_1^{max} / wl_p^{max}$	Priority multiplier
$\alpha > 1$	Urgency amplifier for overdue cases
u_{ce}, κ_{ed}	Equipment need, daily capacity
$\rho(c), los_c, \beta_p$	Bed pool, stay, capacity
π^{ovf}	Bed-overflow charge per day

Priority multipliers derived from SIGIC: $\mu_2 = 270/60 = 4.5$, $\mu_3 = 270/15 = 18$, $\mu_4 = 270/3 = 90$. A tier whose policy allows only 1/18 of P1's wait is, by that same policy, 18× more wait-sensitive.

Decision variables

Presence and scheduling – for every eligible (c, d, r) :

$$pr_{cdr} \in \{0, 1\}, \quad start_{cdr}, end_{cdr} \in [0, k_{dr}]$$

For every non-priority-4 case: $u_c \in \{0, 1\}$, $\sum_{d,r} pr_{cdr} + u_c = 1$

Two interval variables per candidate – the key design choice:

$$iv_{cdr} = \text{NewOptionalIntervalVar}(\text{start}, t_c^{tot}, \text{end}, pr_{cdr}) \quad [\text{ROOM}]$$

$$sgiv_{cdr} = \text{NewOptionalIntervalVar}(\text{start}, t_c^{op}, \text{sgend}, pr_{cdr}) \quad [\text{SURGEON}]$$

Why two intervals?

Surgeon H operates Case X: $t^{op} = 70$, $t^{clean} = 20$, starts at $t = 0$. The *room* stays blocked until $t = 90$. The *surgeon* is free at $t = 70$ and can scrub into Room 2 for Case Y while Room 1 is cleaned. A single t^{tot} -sized interval for C8 would wrongly keep the surgeon blocked until $t = 90$ in every room. Giving C7 (room, size t^{tot}) and C8 (surgeon, size t^{op}) the same start resolves this without any extra channelling constraint.

Objective function

$$\min \underbrace{\sum_{\substack{c: dd_c \geq 0 \\ d \in D_{c,r}}} [dd_c + d] pr_{cdr}}_{\text{Term 1 - on-time, earlier preferred}} + \underbrace{\sum_{\substack{c: dd_c < 0 \\ d \in D_{c,r}}} [dd_c + \alpha d] pr_{cdr}}_{\text{Term 2 - overdue, front-loaded}} + \underbrace{\sum_{c: p_c \neq 4} w_c u_c}_{\text{Term 3 - non-scheduling penalty}} + \underbrace{\pi^{ovf} \sum_{c: \rho(c) \neq \text{none}} \text{ovf}}_{\text{Term 4 - bed overflow}}$$

$$w_c = \mu_{p_c} \cdot \text{PenaltyCurve}(dd_c) + 1.2 \cdot \max_{c' \in C} dd_{c'}$$

- **Term 1:** schedules cases on-time, preferring earlier days.
- **Term 2:** for overdue cases, $\alpha > 1$ makes the planner front-load them to limit further delay.
- **Term 3 (dominant):** w_c is calibrated so the model *always* prefers scheduling over dropping; penalty escalates sharply past the SIGIC deadline and with higher priority.
- **Term 4:** makes bed-overflow cost explicit; forces the model to account for stays crossing the planning horizon.

PenaltyCurve (src/model/penalty.py): flat while the case has slack, escalates sharply on breach, keeps rising with backlog – the shape that makes this objective reproduce SIGIC's intent.

Constraints C1–C6: case disposition and eligibility

C1 – at most one occurrence per patient per week:

$$\sum_{c: \text{patient}(c)=n} \sum_{d \in D_{c,r}} \text{pr}_{cdr} \leq 1 \quad \forall n$$

C2 – priority-4 cases locked to day 1:

$$\sum_r \text{pr}_{c,1,r} = 1 \quad \forall c : p_c = 4 \quad (\text{all other slots forced to 0})$$

C3 – every other case is scheduled exactly once or penalised:

$$\sum_{d \in D_{c,r}} \text{pr}_{cdr} + u_c = 1 \quad \forall c : p_c \neq 4$$

C4–C6 – eligibility (resolved at build time, never solver rows):

- **C4:** room-service roster match (e.g. VASC cases only in VASC rooms).
- **C5:** ambulatory-only rooms reject cases requiring overnight stay.
- **C6:** paediatric-block rule – Friday ENT rooms restricted to age ≤ 8 .

Eligibility runs once at candidate-generation time. No pr_{cdr} is ever created for an ineligible (c, d, r) triple – the main variable-count reduction, costing nothing at solve time.

Constraints C7–C11: time, resources, and beds

C7 – room non-overlap (exact): $\text{AddNoOverlap}(\{\text{iv}_{cdr} : d, r \text{ fixed}\}) \forall d, r$

C8 – surgeon non-overlap (on surgeon's own interval):

$\text{AddNoOverlap}(\{\text{sgiv}_{cdr} : \text{surg}(c) = h, d \text{ fixed}\}) \forall h, d$

C9 – surgeon capacity (daily and weekly):

$$\sum_{c: \text{surg}(c)=h} \sum_r t_c^{op} \text{pr}_{cdr} \leq k_{hd} \forall h, d \qquad \sum_{c,d,r} t_c^{op} \text{pr}_{cdr} \leq k_h \forall h$$

C10 – shared equipment, exact time-overlap check:

$\text{AddCumulative}(\{\text{iv}_{cdr} : u_{ce} = 1, d\}, \mathbf{1}, \kappa_{ed}) \forall e, d$

C11 – recovery/ICU beds (multi-day, with horizon overflow):

- $\text{dayof}_c \leftarrow d_{\text{idx}}$ via $\text{OnlyEnforceIf}(\text{pr}_{cdr} = 1)$.
- $\text{bed}_c = \text{NewOptionalIntervalVar}(\text{dayof}_c, \text{los}_c, \text{dayof}_c + \text{los}_c, 1 - u_c)$
- $\text{AddCumulative}(\{\text{bed}_c : \rho(c) = \rho\}, \mathbf{1}, \beta_\rho) \forall \rho$
- $\text{overflow}_c = \max(0, \text{dayof}_c + \text{los}_c - n_d)$

C10 is the “correct C-arm” constraint – actual time overlap, not a day count. C11 requires a concrete “day of surgery” value: structurally impossible in a day-bucket MIP.

Test instances

Demo instance – 20 cases, 5 rooms, 6 surgeons, 3 services

Service	Rooms	Capacity (min/day)	Surgeons
ENT	2	360 ($\approx$ 6 h)	2
ORTHO	1	450 ($\approx$ 7.5 h)	1
VASC	2	660 ($\approx$ 11 h)	3

Activates *every* constraint family in one run: 3 priority-4 cases (C2), shared C-arm (C10), 2 ICU beds (C11), paediatric-block Friday rule (C6), 3 overdue cases already past their SIGIC deadline.

Medium instance – 200 cases, 12 rooms, 17 surgeons, 5 services

- VASC and NEURO share C-arm equipment; 12% of cases need 1–2 ICU days; priority mix 58/26/13/3%.
- Used to measure the CP-vs-MIP performance gap at scale (Gurobi provides the true MIP optimum for comparison).

Demo instance: CP-SAT vs MIP – results table

```
python main.py -instance demo -benchmark -gap 0.0001
```

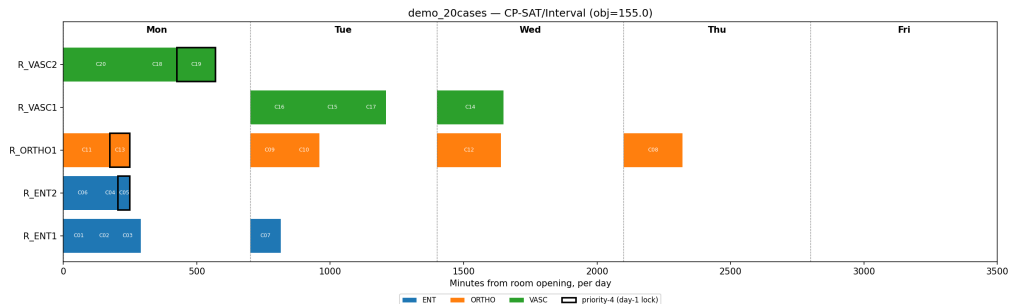
Solver	Status	Objective	Gap	Scheduled
CP-SAT (primary)	Optimal	155.0	0.00%	20/20
OR-Tools/CBC (MIP)	Optimal	157.0	0.00%	20/20
Gurobi (MIP)	Optimal	157.0	0.00%	20/20
CP Optimizer	Optimal	155.0	0.00%	20/20

C-arm cases (C14, C15, C17, C19 – 4 VASC endovascular procedures):

- **CP-SAT:** Mon=[C19], Tue=[**C15, C17**], Wed=[C14] – two C-arm cases placed on the *same* day sequentially.
- **MIP:** Mon=[C19], Tue=[C15], Wed=[C17], Thu=[C14] – one per day, forced by the day-count cap $\sum x_{cdr} \leq 1$.

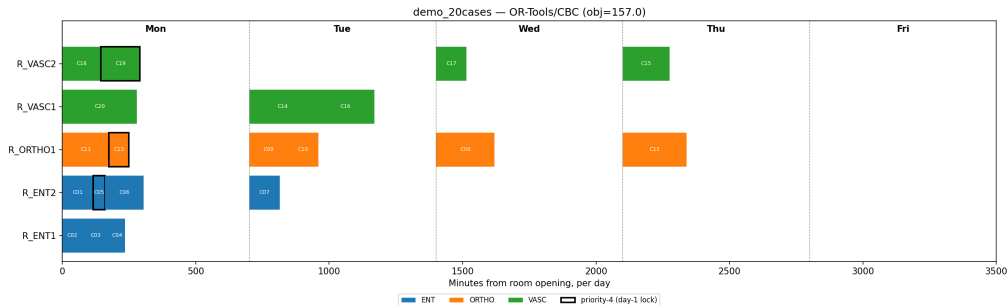
Objective gap (155 vs. 157) comes entirely from this: CP-SAT correctly identifies that the two Tuesday C-arm cases never overlap in clock time, while the MIP's day-count constraint cannot detect that.

Demo – CP-SAT Gantt schedule



Reading the chart: each row = one room; each bar = one case, coloured by service. Bold-outlined bars are priority-4 cases, all correctly placed on Monday. **Tuesday:** R_VASC1 and R_VASC2 each carry one C-arm case sequentially – the model correctly places both on the same day because they never overlap in clock time.

Demo – MIP Gantt schedule (comparison)



Reading the chart: same 20 cases, same services. The MIP produces no exact clock times (day-bucket formulation) – bars are laid out back-to-back for display only; the solver makes no claim about their intra-day order. Compare C-arm spread: Mon, Tue, Wed, Thu each carry exactly one C-arm case. The day-count constraint $\sum x_{cdr} \leq 1$ is satisfied but it prohibits the more efficient two-in-a-day placement CP-SAT finds.

Scaling up: medium instance (200 cases, 2-minute budget)

Step 1 – true MIP optimum (Gurobi, near-zero gap): **74,074.0**, 130/200 scheduled, under 1 second.

Step 2 – realistic 2-minute / 1% gap budget:

Solver	Objective	Own gap	vs. true MIP opt.	Sched.
Gurobi (reference)	74,074.0	0.00%	—	130/200
OR-Tools/CBC	74,116.0	0.56%	+0.06%	130/200
CP-SAT	69,956.0	6.62%	−5.56%	131/200

CBC reaches the MIP's own proven optimum. **CP-SAT beats that proven optimum by 5.56%** while scheduling one additional case – it searches a strictly larger, correct feasible region.

CP-SAT's own 6.62% gap is measured against *its own* larger feasible-region bound – not evidence of poor search quality. The right comparison is the absolute objective value: **69,956 vs. 74,074**.

Medium instance – CP-SAT Gantt (131 cases)



Conclusions

- 1 **CP-SAT finds strictly better schedules on both instances.** Demo: 155.0 vs. 157.0 (MIP proven optimum). Medium: 69,956 vs. 74,074 (MIP's proven optimum) – 5.56% better, one more case scheduled. The cause is always the same: a larger, correct feasible region.
- 2 **Gap percentages mislead without context.** CP-SAT's 6.62% and CBC's 0.56% are measured against *different* feasible-region bounds. The correct comparison is absolute objective value – CP-SAT wins by a wide margin.
- 3 **Shared equipment modelling is the decisive structural factor.** AddCumulative checks actual clock-time overlap; the MIP's day-count cap incorrectly forbids non-overlapping same-day equipment uses.
- 4 **C11 cannot be expressed in a day-bucket MIP.** No variable in that formulation holds “day of surgery” as a value – a structural argument for interval-based CP, independent of performance.
- 5 **The two-interval design reflects real OR practice.** A surgeon free at $t = 70$ can move to a second room while Room 1 cleans until $t = 90$ – a single-interval model incorrectly forbids this.
- 6 **Demo instance: proven optimal in ~ 0.1 s.** Entirely practical for interactive decision support.
- 7 **At a 2-minute budget, CP-SAT already dominates CBC** in both objective quality and cases scheduled.

Questions

Thank you.

► [Back to agenda](#)

► [Full MIP appendix](#)

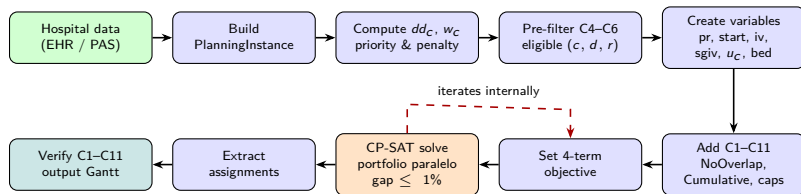
CP-SAT variables – complete formal reference

Variable	Domain	Meaning
pr_{cdr}	$\{0, 1\}$	1 if case c is scheduled day d room r ; created only for eligible (c, d, r) triples (C4–C6 pre-filter)
$start_{cdr}$	$[0, k_{dr}]$	Start time (min from room opening) when $pr_{cdr} = 1$
end_{cdr}	$[0, k_{dr}]$	Room-block end = $start_{cdr} + t_c^{tot}$
iv_{cdr}	OptionalIntervalVar	Room interval: $[start, start + t_c^{tot}]$; active iff $pr_{cdr} = 1$; feeds C7, C10, C11
$sgend_{cdr}$	$[0, k_{dr}]$	Surgeon-end = $start_{cdr} + t_c^{op}$
$sgiv_{cdr}$	OptionalIntervalVar	Surgeon interval: $[start, start + t_c^{op}]$; active iff $pr_{cdr} = 1$; feeds C8 only. Shorter than iv_{cdr} by t_c^{clean}
u_c	$\{0, 1\}$	1 if case c is not scheduled (penalty w_c); only for $p_c \neq 4$; enforced by C3
$dayof_c$	$\{0, \dots, n_d - 1\}$	Index of the day c is scheduled on; derived via OnlyEnforceIf from $pr_{cdr} = 1$; only built for cases with $\rho(c) \neq \text{none}$
bed_c	OptionalIntervalVar	ICU/recovery interval: $[dayof_c, dayof_c + los_c]$; present iff $1 - u_c = 1$; feeds C11
$overflow_c$	$[0, los_c]$	Cumulative Days of bed stay past the horizon; adds Term 4 to objective

CP-SAT search strategy

- **Parallel portfolio:** several complete-search workers with different variable-ordering heuristics, plus Large Neighbourhood Search workers improving an incumbent, all sharing learned clauses via a common SAT core (Perron & Furnon, Google OR-Tools).
- **No custom branching strategy:** OR-Tools' guidance is that the default portfolio outperforms hand-tuned single strategies unless you have structural insight not already exposed through NoOverlap/Cumulative. This model exposes that structure fully.
- **Workers** capped at `os.cpu_count()` – over-subscribing past physical cores adds thread contention, not search diversity.
- **Gap target** = 1% by default; solver stops when the best bound is within 1% of the current incumbent.
- **“Optimal”** means **within the configured gap**, not a literal 0% gap – always printed and never assumed zero.

CP-SAT integration pipeline



Each box maps to a file in the repo. The pre-filter (C4-C6) runs once at build time. The parallel portfolio runs $\leq \text{cpu_count}()$ workers, each sharing learned clauses with the others.

When would MILP be the right choice?

Even within this problem class, MILP is the more natural choice in specific situations:

- **Day-assignment only, no clock times needed.** When the deliverable is “case c runs on day d in room r ” without an exact start time, the day-bucket MIP naturally captures this.
- **No shared resources across rooms.** If each piece of equipment is dedicated to one room and surgeons do not move between rooms, the day-count constraint is sufficient.
- **No multi-day downstream constraint (C11).** When recovery beds are not modelled, one key structural reason for intervals disappears.
- **Small instance, LP relaxation is tight.** A commercial solver (Gurobi, CPLEX) can close the MIP gap in seconds with a provably optimal LP certificate – faster than CP-SAT on small instances.
- **Need LP dual variables.** Shadow prices support sensitivity analysis and what-if reporting – CP-SAT does not expose these.
- **Team expertise is in MILP.** Modelling risk of incorrectly encoding constraints is lower with the tool the team knows best.

MIP – formulation overview

Implemented in `milp_baseline_solver.py`; runs via `-solver milp-cbc` (bundled), `milp-gurobi`, or `milp-cplex` (commercial license required).

Same as CP-SAT: sets, eligibility pre-filter (C4–C6), priority tiers, w_c from the same `penalty.py`.

Key differences:

- No intra-day clock times – decisions at day+room granularity only.
- C7 and C10 are linear capacity sums, not exact time-overlap checks.
- C11 (recovery beds) is *not expressible* – a day-bucket variable holds no “day of surgery” value for a multi-day stay.
- No Term 4 in the objective (C11 does not exist in this model).

MIP – variables

Variable	Domain	Meaning
x_{cdr}	$\{0, 1\}$	1 if case c is scheduled on day d in room r . Only created for eligible (c, d, r) triples. Determines day and room but <i>not</i> exact clock time.
z_c	$[0, 1]$	1 if case c is not scheduled this week (non-priority-4 only). Declared continuous – C3 forces it to $\{0, 1\}$ at the optimum because the objective penalises fractional non-scheduling.

Objective (same three-term structure as CP-SAT, no Term 4):

$$\min \sum_{c: dd_c \geq 0} \sum_{d, r} [dd_c + d] x_{cdr} + \sum_{c: dd_c < 0} \sum_{d, r} [dd_c + \alpha d] x_{cdr} + \sum_{c: p_c \neq 4} w_c z_c$$

MIP – constraints and what's missing

C7 (room, aggregate sum):

$$\sum_{c \in C} t_c^{tot} x_{cdr} \leq k_{dr} \quad \forall d, r$$

For a single room: equivalent to non-overlap. For a *shared resource across rooms*: incorrectly treats all equipment-uses as simultaneous.

C10 (equipment, daily headcount):

$$\sum_{c: u_{ce}=1} \sum_r x_{cdr} \leq \kappa_{ed} \quad \forall e, d$$

Counts *how many* equipment- e cases land on a day, not whether they overlap – the main source of the 5.56% objective gap vs. CP-SAT.

No C11: “ bed_c starts on whichever day c is scheduled” cannot be written – no variable in a day-bucket model holds “day of surgery” as a *value*.

CP Optimizer (optional, appendix)

Kept for one reason: **sequence-dependent room turnover** – a per-adjacent-pair transition matrix via `sequence_var + no_overlap(transition_matrix)`.

	Same service	Different service
CP-SAT (duration-bucketed)	case's own t_c^{clean}	same
CP Optimizer	15 min	35 min

Demo instance: identical to CP-SAT (155.0, 20/20, Optimal).

At 200 cases, own gap closes slower than CP-SAT (search-tuning difference, no warm start applied – not a modelling weakness).

Production deployment

- 1 **Data loader:** map hospital EHR/PAS export to `PlanningInstance`. Schema aligns with Akbarzadeh & Maenhout (2023) CC BY 4.0 datasets – the loader is the only missing piece.
- 2 **Parameter calibration:** replace synthetic defaults with hospital-specific values (wl_p^{max} , surgeon session lengths, bed census, equipment inventory).
- 3 **Weekly cadence:** run with a 30-minute to 2-hour budget. 2-minute results already beat the MIP's proven optimum by 5.56%.
- 4 **Acceptance testing:** extend `tests/test_model.py` with a real-data backtest; any reimplementations must pass the same hard-constraint checks on the demo instance.
- 5 **Fairness monitoring:** track per-service overdue share alongside the aggregate objective (μ_p equity caveat).

Extensions

Extension	Approach
Stochastic durations	Two-stage SP; 1st stage places, 2nd absorbs duration draws
Same-day rescheduling	LNS seeded from current plan
Emergency capacity reserve	Per-day capacity block outside the elective model
Nurse / anaesthetist rostering	Extend H , reuse NoOverlap/cap pattern
Multi-week rolling horizon	Carry forward unscheduled at bumped priority
Sequence-dependent turnover	CP Optimizer transition matrix (appendix)
Per-specialty fairness	Secondary objective bounding per-service overdue share
Real-data pilot	Akbarzadeh & Maenhout CC BY 4.0 datasets

References

- 1 Cardoen, B., Demeulemeester, E., & Beliën, J. (2010). Operating room planning and scheduling: A literature review. *EJOR*, 201(3), 921–932. doi:10.1016/j.ejor.2009.04.011
- 2 Marques, I., & Captivo, M.E. (2015). *Planeamento de cirurgias eletivas no Centro Hospitalar Lisboa Norte*. MSc thesis, Universidade de Lisboa.
- 3 Denton, B.T., Miller, A.J., Balasubramanian, H.J., & Huschka, T.R. (2010). Optimal allocation of surgery blocks to operating rooms under uncertainty. *Oper. Res.*, 58(4), 802–816. doi:10.1287/opre.1090.0791
- 4 SIGIC – Portaria n.º 45/2008, Diário da República, Portugal.
- 5 Van Riet, C., & Demeulemeester, E. (2015). Trade-offs in operating room planning for electives and emergencies. *OR Spectrum*, 37(1), 59–87. doi:10.1007/s00291-013-0345-2
- 6 Macario, A. (2010). What does one minute of operating room time cost? *J. Clin. Anesth.*, 22(4), 233–236. doi:10.1016/j.jclinane.2010.02.003
- 7 Akbarzadeh, B., & Maenhout, B. (2023a). Real life data for OR scheduling problem. Mendeley Data V2. doi:10.17632/n2v49z2vnp.2
- 8 Akbarzadeh, B., & Maenhout, B. (2023b). RealLife OR scheduling dataset, 2021-Jan-May. Mendeley Data V1. doi:10.17632/c8d342266x.1
- 9 Perron, L., & Furnon, V. *CP-SAT: a CP Solver*. Google OR-Tools. developers.google.com/optimization/cp
- 10 Baptiste, P., Le Pape, C., & Nuijten, W. (2001). *Constraint-Based Scheduling*. Kluwer. doi:10.1007/978-1-4615-1479-4
- 11 Vilím, P. (2004). $O(n \log n)$ filtering for unary resource constraints. *CPAIOR 2004*. doi:10.1007/978-3-540-24664-0_15
- 12 Schutt, A., Feydy, T., Stuckey, P.J., & Wallace, M.G. (2009). Why cumulative decomposition is not as bad as it sounds. *CP 2009*. doi:10.1007/978-3-642-04244-7_45
- 13 Laborie, P. (2009). IBM ILOG CP Optimizer for detailed scheduling illustrated on three problems. *CPAIOR 2009*. doi:10.1007/978-3-642-01929-6_5